FIT5230 Milestone 3 交接说明

Dark.MODELBREAKERS · 2026 年 10 月 17 日（周六）· 写给队友

看完这份文档你应该知道：今天做了什么、结果是什么、这些结果怎么讲、接下来 5 天各自要做什么。M3 截止 10 月 22 日 23:55，第 12 周辅导课现场汇报 15 分钟。所有文件都在仓库 github.com/CAVOK1407/FIT5230-Ass1，本地路径 ~/Documents/ChatGPT/FIT5230/github/FIT5230-Ass1。

1. 一句话总结

M3 的技术部分今天基本做完了。我们证明了：在 StealthDiffusion 发布的代码里，论文的核心模块（潜空间对抗优化，LAO）根本到不了存盘的图；存盘图的攻击效果全部来自两次像素 PGD 的输出（控制图）。一次简单的 8/255 像素 PGD 做得和整条流程一样好，失真只有五分之一。M2 的五个开关里只有集成有效，其余四个在判决层面是 0/400——因为它们都作用在到不了输出的那条路上。

现在最缺的不是技术，是和别的队的互动（M3 的 25 分里有 14 分靠这个）、Colab 笔记本（5 分）和幻灯片。

2. 今天做了什么（时间线）

时间	事情	状态
15:30–16:00	读 M2 交接文档和论文草稿；在 m2_stealth.py 上加 M3 需要的全部开关；写运行脚本和报告脚本	完成
16:00	冻结假设：论文第 6、7 节快照放进仓库 paper_frozen/，带 SHA-256，打标签 m3-hypotheses-frozen 推到 GitHub（实验前的时间戳）	完成
15:49–16:05	T1：本地 SD 2.1 的 20 张图上跑 5 个对照配置 + 报告 + 评测；自写 PGD 和 torchattacks 逐像素一致性检查	完成，全部通过
16:20–17:00	造 SD 1.5 数据集（dev 10 / test 100 + 真图）。Mac 连不上 Hugging Face，生成图改在 Colab 做，下回本地	完成
17:00–19:34	T4：SD 1.5 百张上跑 8 个配置 + 像素 PGD 基线 + 6 种后处理的评测	完成
19:40	写结果文档 docs/M3_T1_RESULTS_ZH.md、docs/M3_T4_RESULTS_ZH.md	完成
20:07	收到 PreDecodeGuard 队的回复（同意做潜空间攻击试点）；起草回帖	待发
20:30 起	T2（封顶变体）、T3（期望变换）挂机跑，约 1 小时 40 分钟	进行中

3. 代码：改了什么、怎么跑

还是 M2 的那一个脚本 m2_stealth.py，没有另起炉灶。所有新东西都是命令行开关，默认值等于官方流程（今天重跑的官方配置和 M2 的 B00 逐像素相同，噪声底是 0）。源文件在 m2_scripts/，运行用的副本在 StealthDiffusion/，两边一样。

文件	作用
m2_scripts/m2_stealth.py	攻击主程序。新开关：--encode_source（编码哪张图）、--control_source（控制图来源：pgd / encoded / clean）、--pgd_impl own（自写 PGD 循环）、--cap auto（封顶，T2）、--pgd_eot / --lao_eot（期望变换，T3）、--pgd_only（像素 PGD 基线，T5）。每张图另存 x1（_pgd1.png）和控制图（_control.png）

m2_scripts/m2_evaluate.py	评测。name=dir:后缀 可以直接评 x1、x2 文件；加了 eval 模式断言；resize075 这类条件
m2_scripts/m3_report.py	T1 报告：干净图一致性、分阶段表、配置两两对比、H1(b)
m2_scripts/m3_run.sh	一条命令跑一个阶段：t1 / pgdcheck / data15 / t4 / t2 / t3 / t5 / eval15 / status。每步写日志到 results/m3_logs/，攻击都带 --resume，可以中断续跑
paper_frozen/	论文第 6、7 节的冻结快照 + 哈希

怎么跑（在 Mac 终端）：

cd ~/Documents/ChatGPT/FIT5230/github/FIT5230-Ass1/StealthDiffusion && conda activate fit5230m2 && bash m3_run.sh status

配置名和论文草稿表里的一样：A1–A5 是原论文消融表的五行，X1–X6 是我们加的。常用的几个：

配置	**含义**	**开关**
A5 / B00	官方流程	（默认）
A2	官方流程去掉潜空间优化	--iterations 0
X2	控制图换成干净原图	--control_source clean
X3	去掉第二次 PGD	--control_source encoded
X4	完全不攻击，只编码再解码	--encode_source clean --control_source clean --iterations 0
ENS / FULL	M2 的集成 / 全开	同 M2
PGD8	像素 PGD 基线 8/255	--pgd_only --pgd_impl own --pgd_eps 8/255 --pgd_alpha 2/255

4. 结果

4.1 数据

SD 1.5 生成图 100 张（test）+ 10 张（dev），真实照片 Imagenette 100 + 10。和 M2 在 Colab 上的数据同一套提示词和种子。干净图被四个检测器抓住的张数（也就是条件成功率的分母）：E 94、R 84、D 99、S 98。无损 PNG 下真实图假阳性率四个都接近 0。所有数字都从存盘的 PNG 文件重算，不用内存里的张量（原因见 4.5）。

4.2 主表：无损 PNG，条件成功率

配置	**E**	**R**	**D**	**S**	**均值**	**PSNR**	**L∞**
PGD8 像素 PGD 8/255	1.00	0.80	0.38	0.99	0.79	34.1	0.035
A1 一次 PGD 4/255	1.00	0.48	0.17	0.94	0.65	37.8	0.020
X1 只有控制图	1.00	0.76	0.33	1.00	0.77	34.5	0.039
A2 无潜空间优化	1.00	0.76	0.37	0.91	0.76	34.1	0.183
A5 官方流程 (= B00)	1.00	0.77	0.39	0.90	0.77	34.0	0.183
X3 去掉第二次 PGD	1.00	0.56	0.22	0.86	0.66	35.7	0.177
X2 干净控制图	0.06	0.07	0.00	0.16	0.07	38.6	0.174
X4 完全不攻击	0.01	0.04	0.01	0.15	0.05	39.1	0.172
B10 损失在跳连解码器	1.00	0.76	0.38	0.91	0.76	32.6	0.188

ENS 三代理	1.00	1.00	1.00	0.97	0.99	33.9	0.183
FULL 全开	1.00	1.00	1.00	0.97	0.99	33.4	0.186

A5 的 0.77 和 M2 Colab 上的 B00 完全一样，ENS 0.99 对应 M2 的 1.00，头条数字复现了。

4.3 配置两两对比（100 张 × 4 检测器 = 400 个判决）

对比	回答什么	判决变化
A5 → A2	潜空间优化对存盘图有没有用	4 / 400
B10 → A2	把损失算到跳连解码器上之后呢	7 / 400
A5 → X2	控制图换成干净图	259 / 400
X2 → X4	控制图干净时，潜空间路线带了多少攻击	13 / 400
A5 → X3	去掉第二次 PGD	43 / 400
PGD8 → A5	像素 PGD 8/255 对整条流程	40 / 400（PGD8 不比 A5 差）
ENS → A5	集成的收益	86 / 400
FULL → ENS	边际损失 + LPIPS + 输入多样性 + 跳连路径一起	0 / 400

4.4 后处理（JPEG 75）

官方流程在 JPEG 75 之后，三个迁移检测器上只剩 R 0.15、D 0.02、S 0.06。集成在 R 上还有 0.62（M2 发现的"集成顺带提高鲁棒性"复现了，但只在 R 上）。缩放 0.5 之后 E、R、D 把 96% 以上的真照片判成 AI，那些格子没有意义，汇报时要说明。AUC：E 上所有含 E 的配置都是 0.00（推过头），S 上 0.98–0.99（判决翻了，排序没动），和 M2 第 8 节一致。

4.5 一条教训：以文件为准

S 对浮点张量和 8 位 PNG 的反应不一样。X4（完全不攻击）在内存张量上 S 判 79/100 为真，存成 PNG 再读回来只有 17/100。Control-VAE 的重建在浮点上让 S 偏"真"约 8 分，截成 8 位就没了。E、R、D 没有这个现象。所以评测一律读文件。

5. 这些结果怎么讲

汇报里"框架进展"那 3 分钟的主线：

- M2 说：集成有效、扛不住 JPEG、AUC 说没人真被骗。
- M3 查清了为什么：原论文的核心模块到不了输出；存盘图的攻击成分来自两次像素 PGD 的输出，经一条全分辨率跳连直接进入解码器输出。
- 所以 M2 的五个开关里只有集成（作用在 PGD 阶段）有效，其余四个 0/400——这解释了 M2 当时没解释的 FULL = ENS。
- 一次 8/255 的像素 PGD 不比整条流程差，失真五分之一，快 15 倍。
- T2 把过冲修掉（封顶 PGD），T3 把压缩的洞补上（期望变换），都放在 PGD 阶段，因为输出只看 PGD。

几条规矩：每个数字带分母；不说"论文方法无效"，只说"发布的实现里是这样"；"推过头"和"要报无阈值指标"是已知现象，引 Ozbulak 2019 和 Mavali 2024，我们的新意是在 StealthDiffusion 上第一次量出来；S 的数字带 4.5 节的说明。

6. 同伴互动现状

队伍	我们发了什么	状态	怎么办
PreDecodeGuard-SD3.5（Osama）	后处理对比建议 + 5 个效用攻击提示词	已回复，同意开一条"潜空间访问攻击"试点：给我们留出的潜变量（1×16×128×128 fp16）、冻结的 CNN 权重和阈值、扰动预算；我们返回扰动后的潜变量；他们解码 + 人工审核 + CNN 判定	回帖已写好（见 docs/M3_FOLLOWUP_MESSAGES_EN.md 或聊天记录），今晚发。拿到包后半天做完攻击
Light.Umbrella（DAA-Lite）	5 个控制变量的提示词，要 K=6 和 K=50 的分数	未回复	催一次；没回就按"发了什么、预期什么、没回说明什么"讲
Light.Fahhh（SafeDecoding）	13 个贴边良性提示词 + 4 组对照（回帖写好但可能没发）	未回复	他们有公开 Gradio 页面，不用等回复自己跑，一小时

潜空间攻击的技术方案：对每个潜变量在预算内做 PGD，把 CNN 分数压到阈值以下，到阈值减一个小余量就停（和 T2 的封顶逻辑一样），位移和范数都最小；另交两三个不扰动的对照潜变量测他们流水线的噪声底。这条线就是策略 B 的全部内容，"对方怎么回应"那 5 分也有着落。

7. 接下来 5 天

日期	事情	谁
周六晚（今天）	发 PreDecodeGuard 回帖；催 Umbrella、Fahhh；T2/T3 挂机	王 / 两人定
周日	T2/T3 出表写文档；在 Fahhh 的 Gradio 上跑 13 个提示词和 4 组对照；定分工：谁讲 B（PreDecodeGuard），谁讲 C（Fahhh）	两人
周一	Colab 笔记本：以 M2_Submission.ipynb 为底加 T1–T5 配置和结果表，在 Colab 上从头跑一遍留输出，路径集中、不改官方文件、写明 dev/test 划分和分母；PreDecodeGuard 的包到了就做攻击	一人跑 Colab，一人做攻击
周二	幻灯片 12–14 页，按评分表时间分配（3 + 2 + 5 + 5 分钟）；排练两遍卡表	两人
周三 10/22	交 PDF（放 Colab 链接）到 Moodle，23:55 截止	—

辅导课是周几还没确认。如果在周一或周二，Colab 和幻灯片要提前到辅导课前一天。

8. 文件清单

文件	内容
docs/M3_T1_RESULTS_ZH.md	T1 结果（SD 2.1，20 张）
docs/M3_T4_RESULTS_ZH.md	T4 结果（SD 1.5，100 张），本文第 4 节的完整版
docs/M3_M4_RESERVE_ZH.md	M3/M4 保留项清单（M2 时写的）
docs/M3_FOLLOWUP_MESSAGES_EN.md	给三支队的催帖和汇报记录表
StealthDiffusion/results/m3_sd15/<配置>/	每张图的 records.jsonl、run_config.json 和 PNG；eval/summary.csv 是评测汇总
StealthDiffusion/results/m3_sd21/	T1 的结果，t1_report.md 是自动报告

StealthDiffusion/results/m3_logs/	每个阶段的日志
paper/sections/、paper/README_ZH.md	论文草稿和说明

9. 注意事项

- results/m3_* 下的 PNG、data15/、日志不进仓库（.gitignore 已加），只有 records、run_config、评测 CSV 进。任何表都能从这些文件重算。
- Mac 对 Hugging Face 时通时不通。攻击用的 SD 2.1 权重和检测器权重已在本地缓存，不需要联网；要新下载的东西走 Colab。
- 改代码前先看干净图那一列在各配置间是否一致（m3_report.py 第 0 节），这是最好的一致性检查。
- 假设已经冻结，实验结果和规则不符时照实报"偏离"，不改规则。
- 每次跑完：git add -A && git commit -m "..." && git push。